

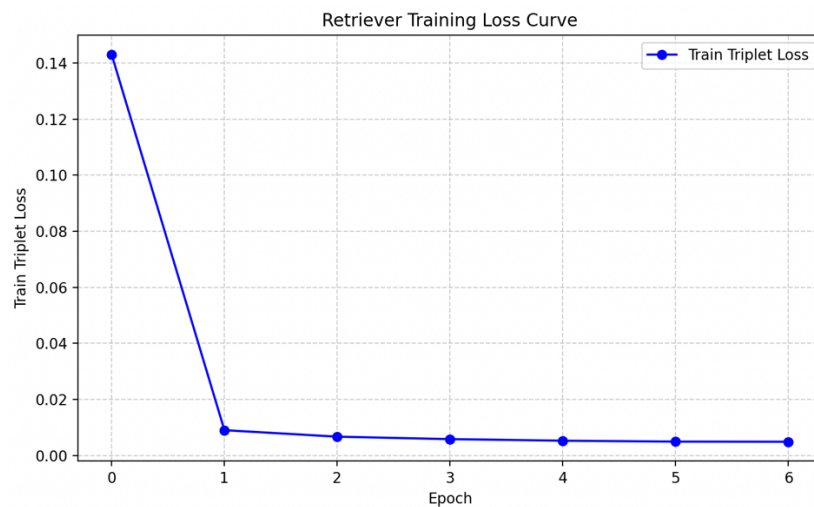
Report

Q1 Retriever & Reranker Tuning

Retriever

1. 在本次實驗中，Retriever 的訓練資料由 train.txt、qrels.txt 和 corpus.txt 組成，主要透過構建 (query, positive, negative) 來訓練模型。每筆資料會從 rewrite 欄位提取查詢作為 anchor，並利用 qrels 中標記為 1 的段落作為正樣本，負樣本則分為兩類：若資料中包含 evidences 且標記為 0，則作為語意相近的難負樣本；若數量不足，則從 corpus 中隨機抽取非正樣本段落補足。最終每個 query 對應一筆 positive 與最多 8 筆 negative。
2. 本次 Retriever 的訓練採用了**雙重損失設計**：Multiple Negatives Ranking Loss (MNR) 與 Triplet Loss。在 **MNR loss** 中，每一個 (query, positive) pair 會取一個 positive，將 batch 內其他 positive 當作負樣本，模型透過最大化正樣本相似度、最小化其他樣本相似度，學會在語意空間中將相關的查詢與段落靠近。這種設計能有效利用大批次資料進行全域對比學習，強化語意一致性。**Triplet loss** 則採用 (query, positive, negative) 進行訓練，以 cosine similarity 為基礎，拉近正樣本、推遠負樣本，其中 margin 設為 0.2。這讓模型在處理語意相近的 hard negatives 時，也能維持穩定的分界。在實作中，每個 epoch 都同時進行 MNR 與 Triplet 訓練，以結合全域與局部語意對比的優勢。
3. 模型訓練 6 個 epoch，max_seq_length 設定為 384。學習率為 3e-5，使用 warmup ratio = 0.1 進行線性學習率升溫，並加入 weight decay = 0.01。Multiple Negatives Ranking Loss 的 batch size 設為 128，Triplet Loss 的 batch size 設為 32，margin 設為 0.2。

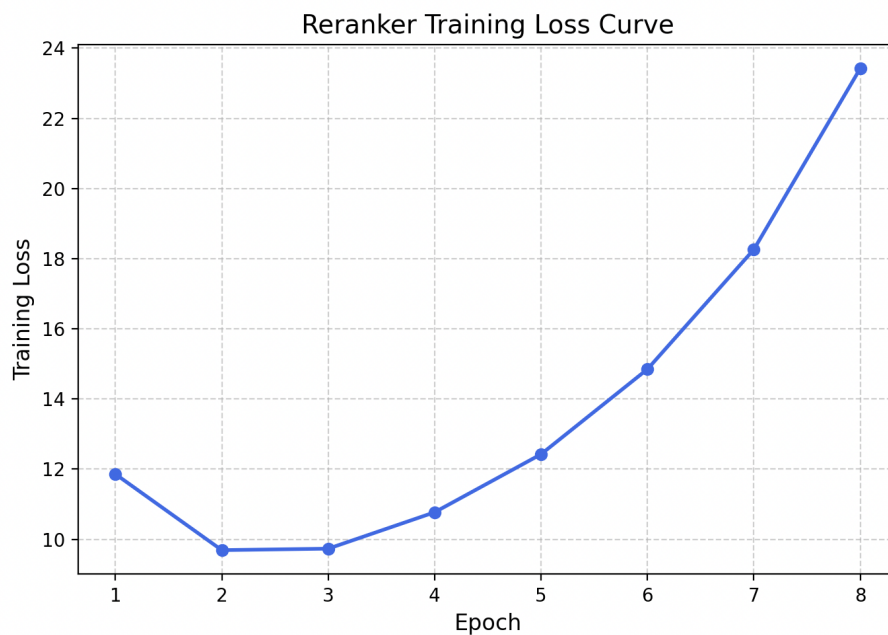
4.



Reranker

1. Reranker 的訓練資料是利用 Retriever 的排序結果搭配 qrels 標註檔建立的，每筆資料由一個查詢（query）和一段段落（passage）組成，並標上 label = 1（相關）或 label = 0（不相關）。在資料建構過程中，先用訓練好的 retriever 模型對所有段落編碼成向量，接著對每個 query 取出前 20 筆（top-K）檢索結果。其中 qrels 檔案中標示的正確段落會作為 positive，而 top-K 中未被標註為正確的段落則當作 hard negative，最多取 5 個負樣本，最終生成的 reranker_train.jsonl 檔案包含許多 (query, passage, label)。
2. Reranker 使用的是 Mean Squared Error, MSE。模型在訓練時會輸入一組 query 和段落 passage，並輸出一個分數表示兩者的相關程度，每個樣本都有一個標籤，1 代表相關，0 代表不相關，MSE 會根據模型預測分數與真實標籤之間的差距來更新權重，使模型逐漸學會讓相關的段落分數更高、不相關的段落分數更低。
3. 模型訓練 8 個 epoch，max_seq_length 設定為 512。learning rate 為 5e-6，使用 warmup ratio = 0.1 進行線性學習率升溫，batch size 設為 48，seed 設定為 42。

4.



Q2: Prompt Optimization

1. Provide a detailed explanation of how you designed your prompt.

以下為 Prompt A 的設計說明

在 System Prompt 中，我設定模型的角色為“a helpful assistant that extracts the exact answer span from provided passages.” 這樣可以讓模型明確知道任務不是自由生成句子，而是要「找出原文中最貼近問題的片段」，後半段的句子：“If no relevant answer is found, respond with only: CANNOTANSWER.”則是為了處理沒有答案的情況，避免模型產生幻覺。

在 User Prompt 的部分，開頭指令 “Extract the exact answer span from the passages below that answers the question” 點出模型的行為目標；接著的 “Only copy the span directly from the passages without any additional explanation or modification” 確保模型只複製原文中的文字，而不進行重組或改寫，最後再重申若 passage 中沒有答案，必須回覆 “CANNOTANSWER”，確保所有輸出格式一致並易於後續自動化評估。

2. Present at least three different prompts you experimented with, along with their inference performance, to demonstrate the effectiveness of your optimization approach.

以下為 Prompt A :

System Prompt : You are a helpful assistant that extracts the exact answer span from provided passages. If no relevant answer is found, respond with only: CANNOTANSWER.

User Prompt : Extract the exact answer span from the passages below that answers the question.

Only copy the span directly from the passages without any additional explanation or modification.

If none of the passages contain the answer, respond with only: CANNOTANSWER.

Passages:

{context}

Question:

{query}

Answer:

```
Recall@10: 0.8447
MRR@10 (after rerank): 0.7502
Bi-Encoder CosSim: 0.3487
```

以下為 Prompt B:

System Prompt : You are an assistant that answers questions based on the provided passages. Try your best to give a helpful answer using the information below.

User Prompt : Please answer the following question based on the given passages.

Passages: {context}

Question: {query}

Answer:

以下為 Prompt C:

System Prompt : You are a knowledgeable assistant who tries to answer questions based on the given passages. Provide an answer that seems relevant and informative.

User Prompt : Use the following passages to help answer the question below.

You may include any information that you think could help form a complete answer.

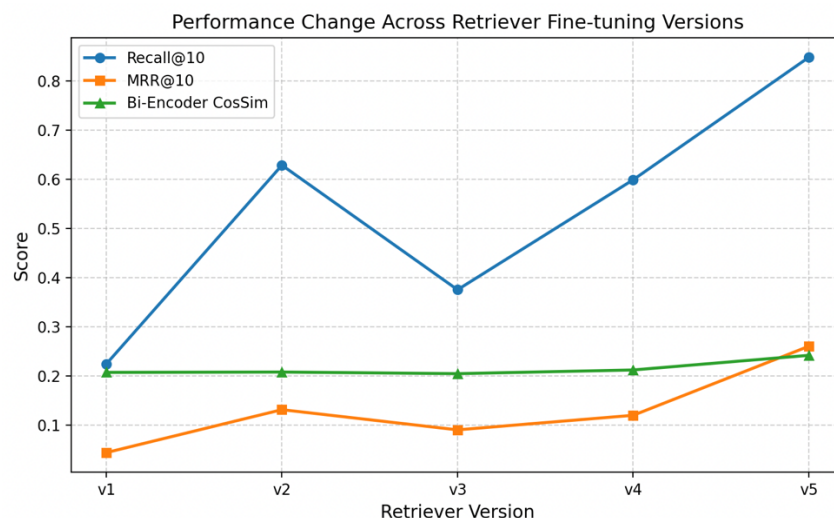
Passages: {context}

Question: {query}

Answer:

Prompt A 嚴格限制模型只能從提供的 passages 中擷取答案片段，並在找不到相關內容時輸出 CANNOTANSWER，這使得模型的回答有最好的表現，Prompt B 與 Prompt C 雖然也要求根據 passages 回答，但缺乏對回答形式與範圍的嚴格限制，導致模型有時會有冗長或不完全正確的情況產生。

Q3: Additional Analysis



我在 Finetune Retriever 的過程中發現 MRR@10 隨著 Recall@10 提升與下降，觀察到 MRR@10 與 Recall@10 呈現明顯的正相關關係，這代表模型在檢索階段的改善，不僅提高了找到正確段落的機率，也讓正確段落的最終排序中被更準確地排序在前。

這一現象可以從推論程式的運作邏輯得到解釋，在 `inference_batch.py` 中，`recall_at_k()` 函式僅判斷前 K 個候選段落中是否包含正確答案，而 `mrr_at_k()` 則依據該答案的實際排名給予分數，當 Retriever 經過 fine-tuning 後，`embedding` 的語意區分能力提升，使得搜尋出的前 K 筆結果更容易包含正確段落，當候選集中有正確答案時，Reranker 才可以將其段落排到前面，MRR 指標跟著上升，如果 Retriever 在前 K 筆檢索中未能包含正確答案，即使 Reranker 表現再好也無法補救。